Multiple Linear Regression using Python - ML

Linear regression is a statistical method used for predictive analysis. It models the relationship between a dependent variable and one or more independent variables by fitting a linear equation to the data. Multiple Linear Regression specifically extends this concept to include two or more independent variables.

Steps for Multiple Linear Regression

Steps to perform multiple linear regression are similar to that of simple linear Regression but difference comes in the evaluation process. We can use it to find out which factor has the highest influence on the predicted output and how different variables are related to each other. Equation for multiple linear regression is:

$$y=\beta0+\beta1X1+\beta2X2+\cdots+\beta nXn$$

Where:

- Y is the dependent variable
- X1,X2,⋯Xn$X1,X2,\cdots Xn$ are the independent variables
- β0$\beta0$ is the intercept
- β1,β2,⋯βn$\beta1,\beta2,\cdots\beta n$ are the slopes

The goal of the algorithm is to find the best fit line equation that can predict the values based on the independent variables. A regression model learns from the dataset with known X and y values and uses it to predict y values for unknown X.

Handling Categorical Data with Dummy Variables

In multiple regression model we may encounter categorical data such as gender (male/female), location (urban/rural), etc. Since regression models require numerical inputs then categorical data must be transformed into a usable form. This is where Dummy Variables used. These are binary variables (0 or 1) that represent the presence or absence of each category. For example:

- **Male**: 1 if male, 0 otherwise
- **Female**: 1 if female, 0 otherwise

GENDER	MALE	FEMALE
Male	1	0
Male	1	0
Female	0	1
Female	0	1
Male	1	0
Female	0	1
Male	1	0

Table

In the case of multiple categories we create a dummy variable for each category excluding one to avoid multicollinearity. This process is called one-hot encoding which converts categorical variables into a numerical format suitable for regression models.

Multicollinearity in Multiple Linear Regression

Multicollinearity arises when two or more independent variables are highly correlated with each other. This can make it difficult to find the individual contribution of each variable to the dependent variable.

To detect multicollinearity we can use:

1. **Correlation Matrix:** A correlation matrix helps to find relationships between independent variables. High correlations (close to 1 or -1) suggest multicollinearity.
2. **VIF (Variance Inflation Factor):** VIF quantifies how much the variance of a regression coefficient increases if predictors are correlated. A high VIF typically above 10 indicates multicollinearity.

Assumptions of Multiple Regression Model

Similar to simple linear regression we have some assumptions in multiple linear regression which are as follows:

1. **Linearity**: Relationship between dependent and independent variables should be linear.
2. **Homoscedasticity**: Variance of errors should remain constant across all levels of independent variables.
3. **Multivariate Normality**: Residuals should follow a normal distribution.
4. **No Multicollinearity**: Independent variables should not be highly correlated.

Implementation of Multiple Linear Regression Model

We will use the **California Housing dataset** which includes features such as **median income, average rooms** and the **target variable**, **house prices**.

Step 1: Importing Libraries

We will be using numpy, pandas, matplotlib and scikit learn for this.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.datasets import fetch_california_housing
```

Step 2: Loading Dataset

- Load the California Housing dataset from sklearn.datasets.
- Dataset contains features such as median income, average rooms stored in X and the target i.e house prices is stored in y.

```
california_housing = fetch_california_housing()

X = pd.DataFrame(california_housing.data, columns=california_housing.feature_names)
y = pd.Series(california_housing.target)
```

Step 3: Selecting Features for Visualization

Choose two features MedInc (median income) and AveRooms (average rooms) to simplify visualization in two dimensions.

```
X = X[['MedInc', 'AveRooms']]
```

Step 4: Train-Test Split

We will use 80% data for training and 20% for testing.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Step 5: Initializing and Training Model

Create a multiple linear regression model using LinearRegression from scikit-learn and train it on the training data.

```
model = LinearRegression()

model.fit(X_train, y_train)
```

Step 6: Finding Intercept and Slopes

After training the model, we can access the intercept and coefficients of the regression equation.

- **model.intercept_ :** β_0 (intercept)
- **model.coef_ :** β_1, β_2 (slopes of MedInc and AveRooms)

```
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

Output:

Intercept: 0.5972677793933272

Coefficients: [0.43626089 -0.04017161]

Step 7: Making Predictions

Using the trained model to predict house prices on the test data.

```
y_pred = model.predict(X_test)
```

Step 8: Visualizing Best- Fit Line in 3D

Plot a 3D graph where blue points represent actual house prices based on MedInc and AveRooms and the red surface shows the best-fit plane predicted by the model. This visualization helps us to understand how these two features influence the predicted house prices.

```
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X_test['MedInc'], X_test['AveRooms'],
           y_test, color='blue', label='Actual Data')

x1_range = np.linspace(X_test['MedInc'].min(), X_test['MedInc'].max(), 100)
x2_range = np.linspace(X_test['AveRooms'].min(), X_test['AveRooms'].max(), 100)
x1, x2 = np.meshgrid(x1_range, x2_range)

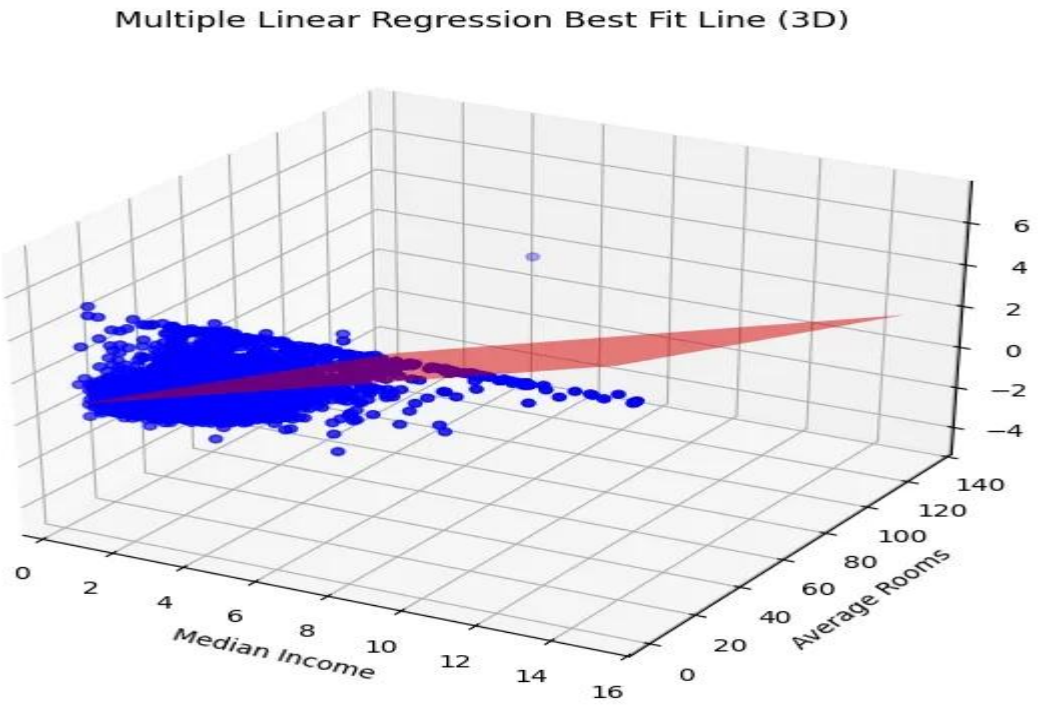
z = model.predict(np.c_[x1.ravel(), x2.ravel()]).reshape(x1.shape)

ax.plot_surface(x1, x2, z, color='red', alpha=0.5, rstride=100, cstride=100)

ax.set_xlabel('Median Income')
ax.set_ylabel('Average Rooms')
ax.set_zlabel('House Price')
ax.set_title('Multiple Linear Regression Best Fit Line (3D)')

plt.show()
```

Output:



Visualizing Multiple Linear Regression

Multiple Linear Regression effectively captures how several factors together influence a target variable which helps in providing a practical approach for predictive modeling in real-world scenarios.